

# LLD

## Low-Level Design (LLD)

### 1. Module-Level Design

#### Document Processing Module

- File: `src/rag\_support/ingest.py`
- Responsibilities:
  - Validate PDF path
  - Load PDF pages via `PyPDFLoader`
  - Normalize metadata (`chunk\_index`, `source\_path`)

#### Chunking Module

- File: `src/rag\_support/ingest.py`
- Uses `RecursiveCharacterTextSplitter`
- Inputs: list of page documents
- Outputs: chunked documents with overlap

#### Embedding Module

- File: `src/rag\_support/ingest.py`
- Function: `build\_embeddings(settings)`
- Uses local sentence-transformer model for chunk embedding

#### Vector Storage Module

- File: `src/rag\_support/ingest.py`, `src/rag\_support/retrieval.py`
- Uses ChromaDB persistent directory
- Stores chunk vectors + metadata

#### Retrieval Module

- File: `src/rag\_support/retrieval.py`
- Class: `KnowledgeRetriever`
- Method: `retrieve(query)`
- Returns:
  - top-k chunks
  - merged context text
  - top confidence score (normalized from vector distance)

#### Query Processing Module

- File: `src/rag\_support/routing.py`
- Responsibilities:
  - intent detection
  - complexity checks
  - route decision (`auto\_answer` or `escalate`)

#### Graph Execution Module

- File: `src/rag\_support/workflow.py`
- Class: `SupportAssistant`
- Uses LangGraph `StateGraph` with `process` and `output` nodes

#### HITL Module

- File: `src/rag\_support/hitl.py`
- Defines callable human responder function

- Default behavior: interactive terminal prompt

## 2. Data Structures

### Document Representation

```
LangChain Document {  
  page_content: str,  
  metadata: {  
    page: int,  
    source: str,  
    chunk_index: int,  
    source_path: str  
  }  
}
```

### Chunk Format

```
{  
  "chunk_id": "42",  
  "page": 3,  
  "score": 0.81,  
  "content": "Refunds are allowed within 14 days ..."  
}
```

### Embedding Structure

- Dense float vector produced by sentence-transformer
- Stored by Chroma as vector + metadata + text payload

### Query-Response Schema

```
{  
  "query": "How do I request a refund?",  
  "intent": "billing",  
  "route": "auto_answer",  
  "confidence": 0.77,  
  "escalation_reason": "sufficient_context",  
  "answer": "You can request a refund within 14 days...",  
  "sources": ["page=1 chunk=12"]  
}
```

### Graph State Object

Implemented as `SupportState` typed dict:

- `query`
- `intent`
- `route`
- `escalation\_reason`
- `confidence`
- `retrieved\_chunks`
- `context`
- `answer`
- `sources`
- `human\_response`

## 3. Workflow Design (LangGraph)

### Nodes

- `process`:
  - retrieve context
  - detect intent

- compute route + reason
- `output`:
  - if route is `auto\_answer`, call Groq with grounded prompt
  - if route is `escalate`, trigger HITL and return human result

## Edges

- `START -> process`
- Conditional edge from `process`:
  - logical branch `answer`
  - logical branch `escalate`
  - both converge to `output` to satisfy the 2-node runtime design
- `output -> END`

## State Propagation

`process` populates state fields used by `output`, especially:

- retrieval context
- confidence
- route decision
- escalation reason

## 4. Conditional Routing Logic

Route decision is deterministic and based on four criteria:

### Answer Generation Criteria (`auto\_answer`)

- Context exists (retrieved chunks > 0)
- Top relevance score >= confidence threshold
- Intent is not sensitive/human-request
- Query not flagged as complex

### Escalation Criteria (`escalate`)

- Low confidence (`top\_score < threshold`)
- Missing context (`no\_relevant\_chunks`)
- Complex query (long, multi-question, or policy-exception marker)
- Intent indicates risk (legal/sensitive/human-request)

## Pseudocode

```
intent = detect_intent(query)
retrieval = retrieve(query)
if intent in {sensitive, human_request}: escalate
else if retrieval.empty: escalate
else if retrieval.top_score < threshold: escalate
else if is_complex_query(query): escalate
else: auto_answer
```

## 5. HITL Design

### When Escalation Is Triggered

- Any escalation criteria matched in routing module.

### What Happens After Escalation

1. Workflow enters `output` node with `route=escalate`
2. Human responder callback is invoked
3. Human answer is attached to final response

## Human Response Integration

- `human\_response` stored in graph state
- final `answer` field includes escalation marker + human content
- allows consistent interface to caller regardless of route

## 6. API / Interface Design

CLI interfaces (implemented in `src/rag\_support/cli.py`):

### Ingest Command

```
python app.py ingest --pdf data/customer_support_kb.pdf --reset
```

Input:

- PDF path
- reset flag

Output:

- pages ingested
- chunks stored

### Ask Command

```
python app.py ask "How do I reset password?"
```

Input:

- query text
- optional human override reply

Output:

- answer text
- route metadata
- confidence score

### Chat Command

```
python app.py chat
```

Interaction flow:

1. user enters query
2. workflow executes
3. answer printed
4. if escalated, human prompt is shown

## 7. Error Handling

### Missing Data

- Missing PDF path -> `FileNotFoundError`
- Missing `GROQ\_API\_KEY` for answering -> runtime error with explicit message

### No Relevant Chunks Found

- Retrieval returns empty result
- Route forced to escalation (`no\_relevant\_chunks`)

### LLM Failure

- Any Groq call error is surfaced by workflow invocation

- Operational recommendation:
  - catch and fallback to escalation in service wrapper (future enhancement)

### **Vector Store State Issues**

- Missing or empty collection naturally yields low/no context
- Routing handles this by escalation rather than hallucinated answer

## **8. Testability Notes**

- Routing logic is pure and unit-tested (`tests/test\_routing.py`)
- End-to-end flow can be manually validated with:
  - `ingest`
  - `ask`
  - escalation path queries